Week 7, part 1 of 2

Agents and tool use

What is an agent?

"Agents are systems that independently accomplish tasks on your behalf."[1]

Models decide intention, code executes.

1. OpenAI, *A Practical Guide to Building Agents*, p. 4. https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf ↩

Workflow or agent?

- **Workflow**: "systems where LLMs and tools are orchestrated through predefined code paths."[1]
- **Agent**: "systems where LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks."

1. Anthropic, "Building Effective Agents", December 2024. https://www.anthropic.com/engineering/building-effective-agents ↩

Agent Loop

An agent is a model in a loop:

1. Model reads the context.
2. Model picks a tool call, or decides it is done.
3. Your runtime executes the call.
4. The result is appended to the context.
5. Go to step 1.

Do you need an agent?

"For many applications, however, optimizing single LLM calls with retrieval and in-context examples is usually enough."[1]

"Agentic systems often trade latency and cost for better task performance, and you should consider when this tradeoff makes sense."

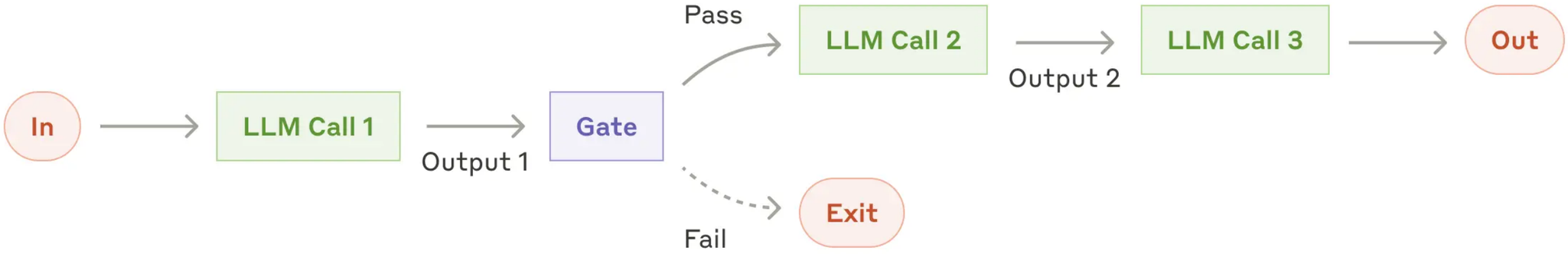
1. Anthropic, "Building Effective Agents", December 2024. https://www.anthropic.com/engineering/building-effective-agents ↩

Workflows

– **Prompt chaining**: fixed sequence, each call feeds the next.
– **Routing**: classify, then dispatch to a specialised path.
– **Parallelisation**: fan out, then aggregate.
– **Orchestrator-workers**: one model plans, others execute.
– **Evaluator-optimiser**: one model generates, another critiques.[1]

1. Anthropic, "Building Effective Agents", December 2024. https://www.anthropic.com/engineering/building-effective-agents ↩

Prompt chaining

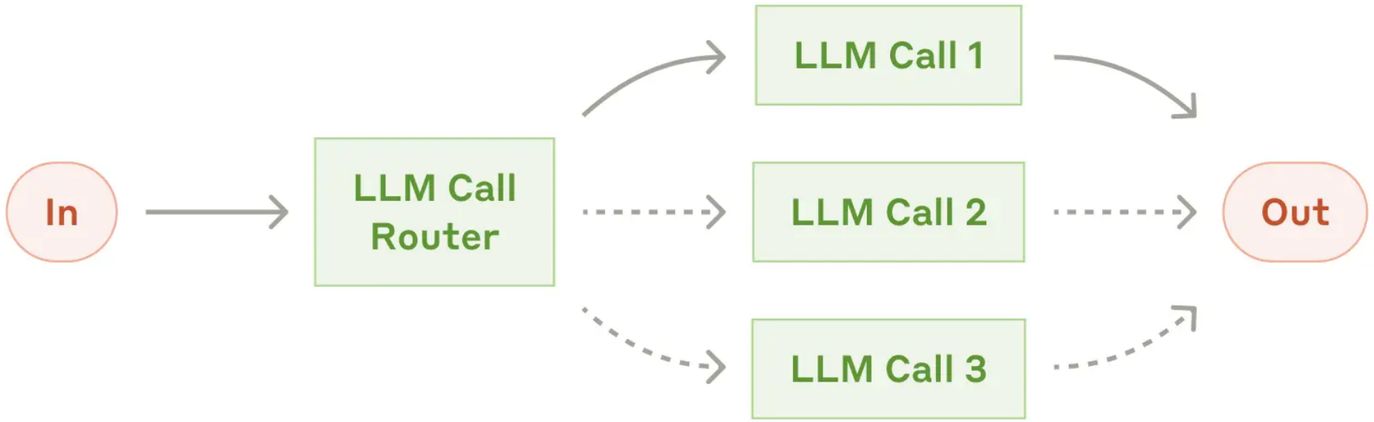


Prompt Chaining, https://www.anthropic.com/engineering/building-effective-agents

PROMPT CHAINING EXAMPLES:

- Generating Marketing copy, then translating it into a different language.
- Writing an outline of a document, checking that the outline meets certain criteria, then writing the document based on the outline.
- Generating social media content and checking it against safety standards

Routing

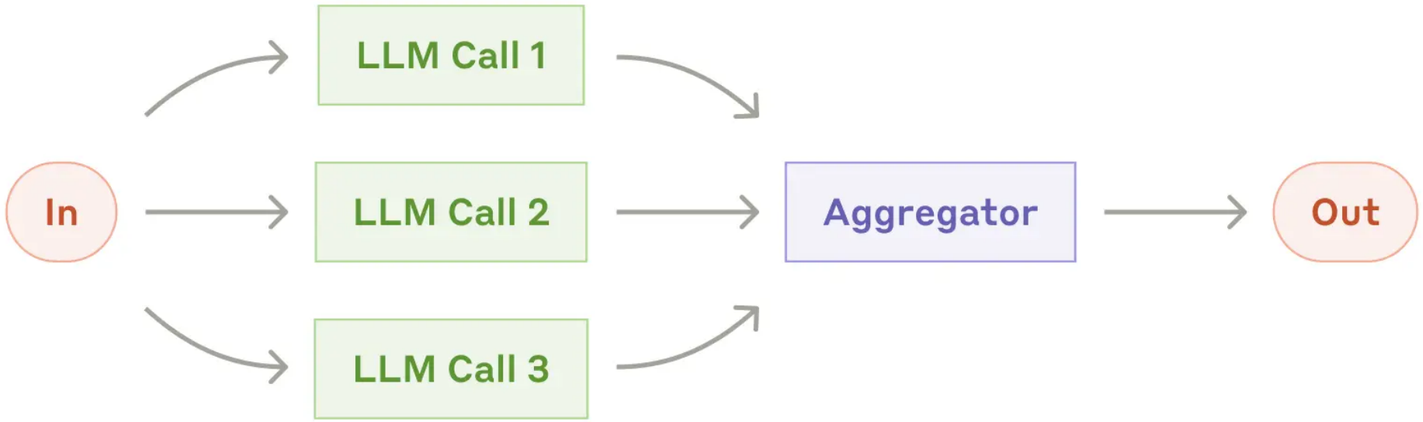


Routing, https://www.anthropic.com/engineering/building-effective-agents

ROUTING EXAMPLES:

- Sorting customer service queries (general questions, refunds, technical support) into different downstream prompts and tools.
- Sending easy, common questions to a small, cheap model and hard, unusual ones to a capable model.

Parallelisation



Parallelisation, https://www.anthropic.com/engineering/building-effective-agents

PARALLELISATION EXAMPLES:

- One call answers the user while a second screens the query for inappropriate content (sectioning).
- Evals where each call scores a different aspect of one output (sectioning).
- Several different prompts review the same code for vulnerabilities, flagging anything any of them finds (voting).

Orchestrator-workers

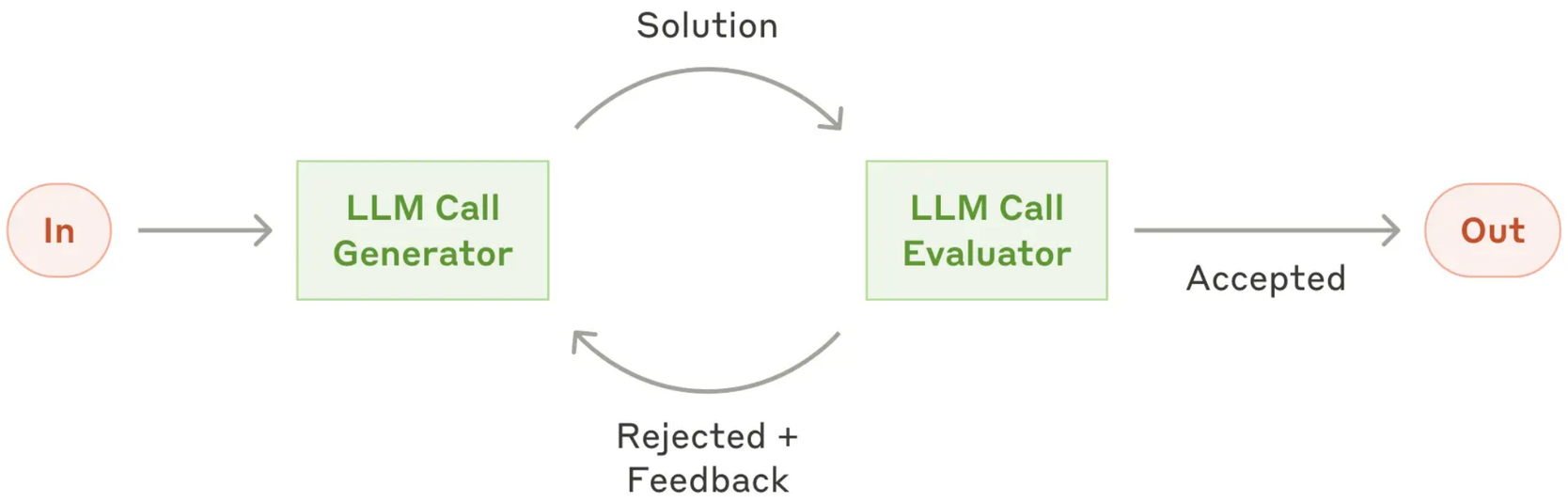


Orchestrator-Workers, https://www.anthropic.com/engineering/building-effective-agents

ORCHESTRATOR-WORKERS EXAMPLES:

- Coding tools that change several files in one task, where which files depends on the task.
- Search that gathers and analyses information from multiple sources.

Evaluator-optimiser



Evaluator-Optimiser, https://www.anthropic.com/engineering/building-effective-agents

EVALUATOR-OPTIMISER EXAMPLES:

- Literary translation, where a critic model catches nuance the translator's first pass missed.
- Multi-round search, where the evaluator decides whether the gathered information is complete or another round is needed.

CONNECTING MODELS TO TOOLS

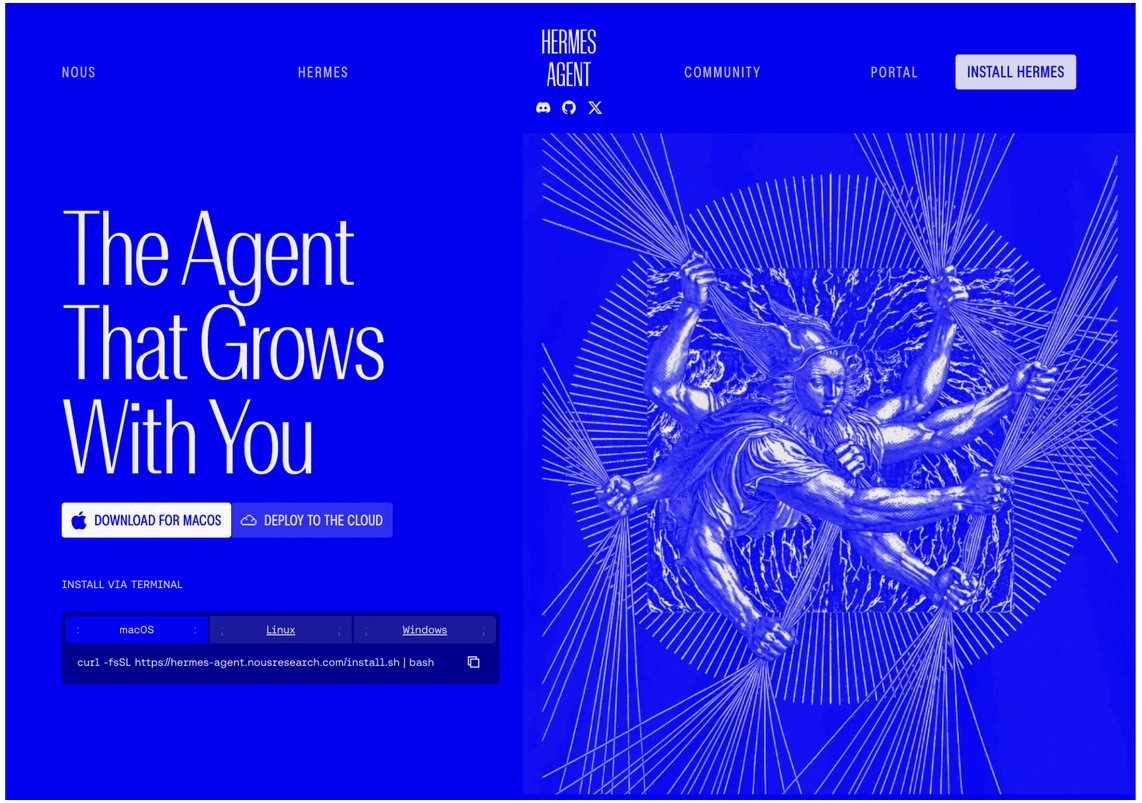
The model emits a structured request (usually json) and a runtime executes and returns the result.

A tool is a schema

```
{
  "name": "get_order",
  "description": "Look up an order by id. Returns the order status, items, and delivery
date. Use when the user asks about an existing order.",
  "input_schema": {
    "type": "object",
    "properties": {
      "order_id": {
        "type": "string",
        "description": "The order id, e.g. ORD-10394"
      }
    },
    "required": ["order_id"]
  }
}
```

Demo

Hermes on a VPS, searching property listings.



Eighteen of twenty-eight Evans Avenue, Eastlakes last sold for $620,000 in September 2024 and is now listed at $425,000. Thirteen of one hundred and seven Wentworth Street, Randwick last sold for $1,135,000 in October 2024 and is now listed at $900,000. Two Waterloo units at 351 George Street appear in the top ten with value gaps above 26%.

#	Property	Asking	Implied value	Gap	Basis
1	**202/1-3 Link Rd** Zetland · 1 bed · 47d listed	$619,000	$850,000	**+37%**	based on recent sales in the same building
2	**211/15 Ralph St** Alexandria · 1 bed · 39d listed	$820,000	$1,312,000	**+60%**	based on recent sales in the same building
3	**543/351 George Street** Waterloo · 1 bed · 38d listed	$720,000	$990,000	**+38%**	based on recent sales in the same building
4	**642/351 George Street** Waterloo · 1 bed · 46d listed	$785,000	$990,000	**+26%**	based on recent sales in the same building
5	**1008/25 Geddes Avenue** Zetland · 1 bed · 41d	$735,000	$1,164,000	**+58%**	based on recent sales in the same building
[illegible]	[illegible]	[illegible]	[illegible]000	**+43%**	based on recent sales in the same building
	Street Waterloo · 1 bed				

DESIGNING A TOOL CATALOGUE

Three kinds of tools:[1]

- **Data**: read context the workflow needs. Query a database, read a PDF, search the web.
- **Action**: change something. Send an email, update a record, hand off to a human.
- **Orchestration**: other agents, wrapped as tools.

1. OpenAI, *A Practical Guide to Building Agents*, p. 9, "Defining tools". https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf ↩

How many tools?

"Some implementations successfully manage more than 15 well-defined, distinct tools while others struggle with fewer than 10 overlapping tools."[1]

1. OpenAI, *A Practical Guide to Building Agents*, p. 16. https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf ↩

Make the wrong call hard

Poka-yoke your tools: "Change the arguments so that it is harder to make mistakes."[1]

1. Anthropic, "Building Effective Agents", December 2024, Appendix 2, "Prompt engineering your tools". https://www.anthropic.com/engineering/building-effective-agents ↩

Poka-yoke example from Anthropic's computer-use

`cwd` commands caused relative paths to resolve with the wrong file edited

```
agent: bash("cd tests/")
agent: edit("utils.py")    # → tests/utils.py, not src/utils.py
```

The fix was to block relative paths and provide feedback to model.[1]

```
def validate_path(self, command: str, path: Path):
    if not path.is_absolute():
        suggested_path = Path("") / path
        raise ToolError(
            f"The path {path} is not an absolute path, "
            f"it should start with `/`. Maybe you meant {suggested_path}?"
        )
```

1. anthropics/claude-quickstarts, computer-use-demo/computer_use_demo/tools/edit.py, validate_path (excerpt trimmed). https://github.com/anthropics/claude-quickstarts/blob/40a7a1446523254f5ec0f3fa6aa873162a1dbbb2/computer-use-demo/computer_use_demo/tools/edit.py ↩

Poka-yoke example #2

SWE-agent's edit tool lints after every edit:[1]

```
pre_edit_lint = flake8(wf.path)
wf.set_window_text(replacement_text, ...)
post_edit_lint = flake8(wf.path)
...
if new_flake8_output:
    # Show error and revert changes
    wf.undo_edit()
```

1. SWE-agent/SWE-agent, tools/windowed_edit_linting/bin/edit. https://github.com/SWE-agent/SWE-agent/blob/3ea751c087f32b16e039a2233dd6eefecef325d5/tools/windowed_edit_linting/bin/edit Yang et al., "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering", NeurIPS 2024, §2. https://arxiv.org/abs/2405.15793 ↩

AGENT FAILURE

The visible error is often downstream of the mistake.

For example, an coding error in an SE agent won't be detected until much later during listing or execution.

Last good state, first bad state

1. Find the **last good state**: the latest point where the context was still correct.
2. Find the **first bad state**: the first action or result that made it wrong.
3. Fix there. Everything after it is contaminated.